

03-which-image-models-are-best

May 27, 2025

The data, concept, and initial implementation of this notebook was done in Colab by Ross Wightman, the creator of timm. I (Jeremy Howard) did some refactoring, curating, and expanding of the analysis, and added prose.

0.1 timm

[PyTorch Image Models](#) (timm) is a wonderful library by Ross Wightman which provides state-of-the-art pre-trained computer vision models. It's like Huggingface Transformers, but for computer vision instead of NLP (and it's not restricted to transformers-based models)!

Ross has been kind enough to help me understand how to best take advantage of this library by identifying the top models. I'm going to share here so of what I've learned from him, plus some additional ideas.

0.2 The data

Ross regularly benchmarks new models as they are added to timm, and puts the results in a CSV in the project's GitHub repo. To analyse the data, we'll first clone the repo:

```
[1]: ! git clone --depth 1 https://github.com/rwightman/pytorch-image-models.git
      %cd pytorch-image-models/results
```

```
Cloning into 'pytorch-image-models'...
remote: Enumerating objects: 454, done.
remote: Counting objects: 100% (454/454), done.
remote: Compressing objects: 100% (391/391), done.
remote: Total 454 (delta 98), reused 238 (delta 59), pack-reused 0 (from 0)
Receiving objects: 100% (454/454), 2.84 MiB | 5.92 MiB/s, done.
Resolving deltas: 100% (98/98), done.
/workspaces/course22/pytorch-image-models/results

/home/vscode/.local/lib/python3.10/site-packages/IPython/core/magics/osm.py:417:
UserWarning: using dhists requires you to install the `pickleshare` library.
  self.shell.db['dhists'] = compress_dhists(dhists)[-100:]
```

Using Pandas, we can read the two CSV files we need, and merge them together.

```
[2]: import pandas as pd
      df_results = pd.read_csv('results-imagenet.csv')
      df_results['model'] = df_results['model'].str.split('.').str[0] # split string
      ↪for new format
```

We'll also add a “family” column that will allow us to group architectures into categories with similar characteristics:

Ross has told me which models he's found the most usable in practice, so I'll limit the charts to just look at these. (I also include VGG, not because it's good, but as a comparison to show how far things have come in the last few years.)

```
[3]: def get_data(part, col, test):
    df = pd.read_csv(f'benchmark-{part}-amp-nhwc-{test}-rtx3090.csv') #_
    ↪adjusted filename
    df = df.merge(df_results, on='model')
    df['secs'] = 1. / df[col]
    df['family'] = df.model.str.extract('^(?([a-z]+)?(?:v2)?(?:\d|_|$)')
    df = df[~df.model.str.endswith('gn')]
    df.loc[df.model.str.contains('in22'), 'family'] = df.loc[df.model.str.
    ↪contains('in22'), 'family'] + '_in22'
    df.loc[df.model.str.contains('resnet.*d'), 'family'] = df.loc[df.model.str.
    ↪contains('resnet.*d'), 'family'] + 'd'
    return df[df.family.str.contains('^re[sg]netd?
    ↪|beit|convnext|levit|efficient|vit|vgg')]

[4]: df = get_data('infer', 'infer_samples_per_sec', 'pt113-cu117')
```

0.3 Inference results

Here's the results for inference performance (see the last section for training performance). In this chart:

- the x axis shows how many seconds it takes to process one image (**note:** it's a log scale)
- the y axis is the accuracy on Imagenet
- the size of each bubble is proportional to the size of images used in testing
- the color shows what “family” the architecture is from.

Hover your mouse over a marker to see details about the model. Double-click in the legend to display just one family. Single-click in the legend to show or hide a family.

Note: on my screen, Kaggle cuts off the family selector and some plotly functionality – to see the whole thing, collapse the table of contents on the right by clicking the little arrow to the right of “Contents”.

```
[5]: import plotly.express as px
    w,h = 1000,800

    def show_all(df, title, size):
        return px.scatter(df, width=w, height=h, size=df[size]**2, title=title,
            x='secs', y='top1', log_x=True, color='family', hover_name='model',_
            ↪hover_data=[size])

[6]: show_all(df, 'Inference', 'infer_img_size')
```

That number of families can be a bit overwhelming, so I'll just pick a subset which represents a single key model from each of the families that are looking best in our plot. I've also separated convnext models into those which have been pretrained on the larger 22,000 category imagenet sample (`convnext_in22`) vs those that haven't (`convnext`). (Note that many of the best performing models were trained on the larger sample – see the papers for details before coming to conclusions about the effectiveness of these architectures more generally.)

```
[7]: subs = 'levit|resnetd?|regnetx|vgg|convnext.*|efficientnetv2|beit'
```

In this chart, I'll add lines through the points of each family, to help see how they compare – but note that we can see that a linear fit isn't actually ideal here! It's just there to help visually see the groups.

```
[8]: def show_subs(df, title, size):
    df_subs = df[df.family.str.fullmatch(subs)]
    return px.scatter(df_subs, width=w, height=h, size=df_subs[size]**2,
        title=title,
        trendline="ols", trendline_options={'log_x': True},
        x='secs', y='top1', log_x=True, color='family', hover_name='model',
        hover_data=[size])
```

```
[9]: show_subs(df, 'Inference', 'infer_img_size')
```

From this, we can see that the *levit* family models are extremely fast for image recognition, and clearly the most accurate amongst the faster models. That's not surprising, since these models are a hybrid of the best ideas from CNNs and transformers, so get the benefit of each. In fact, we see a similar thing even in the middle category of speeds – the best is the ConvNeXt, which is a pure CNN, but which takes advantage of ideas from the transformers literature.

For the slowest models, *beit* is the most accurate – although we need to be a bit careful of interpreting this, since it's trained on a larger dataset (ImageNet-21k, which is also used for *vit* models).

I'll add one other plot here, which is of speed vs parameter count. Often, parameter count is used in papers as a proxy for speed. However, as we see, there is a wide variation in speeds at each level of parameter count, so it's really not a useful proxy.

(Parameter count may be useful for identifying how much memory a model needs, but even for that it's not always a great proxy.)

```
[10]: px.scatter(df, width=w, height=h,
    x='param_count_x', y='secs', log_x=True, log_y=True,
    color='infer_img_size',
    hover_name='model', hover_data=['infer_samples_per_sec', 'family']
)
```

0.4 Training results

We'll now replicate the above analysis for training performance. First we grab the data:

```
[11]: tdf = get_data('train', 'train_samples_per_sec', 'pt112-cu113')
```

Now we can repeat the same *family* plot we did above:

```
[12]: show_all(tdf, 'Training', 'train_img_size')
```

...and we'll also look at our chosen subset of models:

```
[13]: show_subs(tdf, 'Training', 'train_img_size')
```

Finally, we should remember that speed depends on hardware. If you're using something other than a modern NVIDIA GPU, your results may be different. In particular, I suspect that transformers-based models might have worse performance in general on CPUs (although I need to study this more to be sure).

```
[ ]:
```